Position-Aware Structural Knowledge Sharing-Based Federated Graph Learning for Intelligent Transportation Systems

Cheng Dai, Guangdong He, Bing Guo, Sahil Garg, *Senior Member, IEEE*, Georges Kaddoum, *Senior Member, IEEE*, and Mohammad Mehedi Hassan, *Senior Member, IEEE*

***Abstract*— Federated Graph Learning (FGL) combines the powerful graph data modeling capabilities of Graph Neural Networks (GNNs) with the distributed processing requirements of intelligent transportation systems (ITS), making it a prominent focus in recent research. In the context of the Internet of Everything (IoE), diverse devices and services generate highly heterogeneous, non-independent, and non-identically distributed (non-IID) data, which limits model generalization and training efficiency. To tackle these challenges, this paper proposes a Position-Aware Structural Sharing Federated Graph Learning Framework tailored for ITS applications. This framework enhances GNNs' capacity to process cross-domain graph data, significantly improving model applicability and performance across various ITS scenarios. Specifically, we use a structural encoder alongside a position-aware structural encoder to capture generic structural knowledge, sharing these embeddings across clients in an FGL setup. Extensive experiments on different datasets demonstrate that our method outperforms existing approaches in handling non-IID federated graph learning tasks, particularly within cross-dataset and cross-domain environments.**

***Index Terms*— Federated graph learning, graph neural network, Internet of Everything, non-IID data.**

I. Introduction

With advancements in artificial intelligence, Graph Neural Networks (GNNs, detailed in Section II-A) have emerged as a pivotal technology in Internet of Everything (IoE) applications, particularly within intelligent transportation systems (ITS). However, the decentralized nature of IoE data, spread across numerous sensors, limits the efficiency of traditional GNN training. FGL addresses this challenge by enabling collaborative GNNs training in a distributed framework. This approach allows multiple clients to build a shared model without exchanging local data, enhancing computational efficiency and safeguarding data privacy. FGL efficiently transforms graph nodes into low-dimensional embeddings, capturing inter-node relationships and facilitating tasks such as link prediction and graph classification. This makes FGL highly suitable for ITS, where data-driven insights are essential for optimizing traffic management and ensuring system resilience.

In ITS, data generation is highly decentralized and usually exhibits non-IID characteristics. This diversity increases the complexity of FGL due to the different data generated by devices and sensors in different environments, and in particular, the differences in the feature space and connectivity patterns of the graph data significantly affect the aggregation effect and performance of the models [1], [2]. To address these challenges, researchers are exploring various strategies to minimize the impact of heterogeneity in graph data. Although feature information from different graph domains tends to exhibit a high degree of heterogeneity, the underlying features of the graph structure actually have a certain degree of generality across domains, which can be exploited to enhance the adaptability and robustness of the model [3], [4].

To effectively explore the deep knowledge, Dwivedi firstly proposed an innovative approach where a GNN model with decoupled structure and features is trained on a local client, which can effectively separate the structural information from the heterogeneous features from different views [5]. Moreover, FedStar is proposed based on this idea, and it takes the sharing mechanism of this structural knowledge and thus strengthens the effectiveness of the graph deep model when dealing with cross-domain graph data [6]. In fact, FedStar still has some shortcomings in capturing the positional information of nodes in the graph, and the structure-aware capability is also crucial for understanding the relative position-awareness and relationships between nodes in the graph. For example, even if two nodes have very different positions in the graph, if they share the same local neighborhood structure, the existing

Received 6 November 2024; revised 23 February 2025 and 31 March 2025; accepted 10 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U2268204, Grant 62172061, Grant 62202319, and Grant 62302082; and in part by King Saud University, Riyadh, Saudi Arabia, through Ongoing Research Funding Program (ORF-2025-18). The Associate Editor for this article was T. R. Gadekallu. *(Corresponding author: Bing Guo.)*

Cheng Dai, Guangdong He, and Bing Guo are with the College of Computer Science, Sichuan University, Chengdu 610065, China (e-mail: daicheng@scu.edu.cn; 2022223045168@stu.scu.edu.cn; guobing@scu.edu.cn).

Sahil Garg is with the Electrical Engineering Department, École de Technologie Supérieure, Montréal, QC H3C 1K3, Canada, also with the Department of Computer Engineering and Computational Sciences (CECS), School of Engineering, Applied Science, and Technology, Canadian University, Dubai, and also with the Centre for Research Impact and Outcome, Chitkara University Institute of Engineering and Technology, Chitkara University, Rajpura, Punjab 140401, India (e-mail: sahil.garg@ieee.org).

Georges Kaddoum is with the Electrical Engineering Department, École de Technologie Supérieure, Montréal, QC H3C 1K3, Canada (e-mail: georges.Kaddoum@etsmtl.ca).

Mohammad Mehedi Hassan is with the Department of Information Systems, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia (e-mail: mmhassan@ksu.edu.sa).

Digital Object Identifier 10.1109/TITS.2025.3561244

GNN architectures will map them to the same point in the embedding space. This is mainly because the main limitation of existing GNN frameworks is the inability to capture the positional information of graph nodes.

To address this limitation, recent studies have enriched the structural and positional information of GNNs by introducing heuristic strategies. For a simple example, P-GNN [7] effectively overcomes the limitations of GNNs in positional representation by randomly harvesting anchors from the graph, and then generating node positional information encoding. It provides a unique position embedding for each node by utilizing the relative position information from anchor points to other nodes, which enhances the model's representation of node position differences in the graph. However, the strategy of randomly selecting anchor points in P-GNN may ignore some important graph structural features or node relationships, which will affect the model's in-depth understanding and accuracy of the graph data.

To overcome the challenges above, we propose the Position-Aware Structural Sharing Federated Graph Learning Framework. Firstly, we utilize degree-based structural embedding and random wandering-based structural embedding to capture the local basic structural information of the graph. Secondly, we obtain the embedded representations of the nodes based on the positional information of the nodes in the whole network, and then we share the captured structural information among different clients via graph federation. Thus, considering that these structural information do not depend on any domain-specific feature information but are based on the structural properties of the graph, it enables our framework to handle different graph domains in federated learning. Besides that, it can utilize these universal structural features to improve the effectiveness in multiple domains, and thus effectively solving the prevailing Non-IID problem in federated learning.

Our main contributions are summarized as follows:

- We optimize the selection strategy for the set of position-aware anchors with a centralized selection approach, which can help to select optimal anchors for the further feature capturing.
- We effectively overcome the limitations of existing GNN frameworks in capturing node position information by sharing position-aware node embeddings.
- We have conducted extensive experiments, and the results show that our method consistently outperforms the baseline substantially.

We organized the following paper as follows: Section II reviews related work in graph neural networks, federated learning, and federated graph learning. Section III details our proposed methods and key aspects. Section IV contains experiments and analysis of results. Finally, Section V gives the final conclusion and suggests future research directions.

II. Related Works

A. Graph Neural Networks

Graph Neural Networks (GNNs) constitute an entire family of neural network models designed specifically for graph data structures [8]. Unlike traditional neural networks, GNNs are unique in their ability to directly process graph-structured data and to take into account the topological relationships among nodes. Most of the existing GNN models are based on graph messaging architectures that integrate feature information from their neighbors in the graph by applying different aggregation schemes to the nodes. For example, Graph Convolutional Networks (GCNs) employ a representation of average neighbor nodes for information aggregation, while Graph Isomorphic Networks (GINs) aggregate features from neighbor nodes via a summation function [6]. However, all these models focus on capturing the local network structure around a given node, and the structural location encoding extracted by these models performs poorly on graph data spanning multiple domains [9], [10]. To capture a more generalized structural pattern in graph data spanning different domains, we introduce a position-aware network that captures the relative position of a node across the graph [7]. This approach allows the model to learn not only the local neighborhood characteristics of a node, but also to understand the global position of a node in the overall graph structure, thus improving the performance of cross-domain graph data processing.

B. Federated Learning

Federated Learning (FL), a decentralized learning algorithm, is receiving increasing attention due to its ability to achieve distributed collaborative training while protecting user privacy [11], [12], [13]. In FL, data is distributed across remote clients and models are trained cooperatively under the coordination of a central server. This approach effectively solves the problems of privacy leakage and data silos faced by traditional centralized data processing, and also makes full use of the data resources distributed across different clients to achieve global model optimization. The standard FL algorithm trains a local model, calculates local updates based on its own data, and then uploads the local updates to the server, where the central server averages the global parameters. The updated global model parameters are then distributed back to each client for the next round of training. Recently, there have been many FL algorithms developed and the most representative FL algorithm is FedAvg [14]. It performs global parameter averaging at the central server, and multiple experiments have shown that when the data from each client is statistically heterogeneous, which is also one of the pending challenges for existing federated learning [15], [16], [17]. To solve this problem, Zhu proposes different data sharing strategies to solve the problem of data heterogeneity [18]. Fallah proposes personalized federated learning using meta-learning (MAML) for local models [19], while FCCL [20] learns generalized representations by constructing cross-correlation matrices and utilizing unlabeled public data. In addition, CHAFL [21] effectively copes with data heterogeneity in federated learning by taking into account the statistical and systematic heterogeneity of the clients. Besides that, pFedGraph [22] efficiently adapts to the level of data heterogeneity and resists model contamination attacks by learning cooperation. pFedCG [23] efficiently addresses the problem of statistical heterogeneity in joint

learning by using Graph Convolutional Networks to cluster the based knowledge sharing.

C. Federated Graph Learning

Applying the advantages of federated learning to graphs, i.e., FGL, is an emerging research direction. In FGL, each client has its own characteristics and connections, and model training and optimization are performed by means of federated learning. It aims to protect the privacy information of nodes and edges in the graph, while achieving global optimization of the model. However, due to the complexity of the graph data, the nodes or edges in the graph need to be represented as low-dimensional vectors (node embeddings) for better access to valuable information [24]. To achieve this goal, researchers have proposed various solutions to solve the problem of data heterogeneity (non-independent and non-homogeneous distribution) present in FGL. Chen investigated the heterogeneity and complementarity of graphs between clients in the global self-supervised FL framework in the case of non-independent and non-homogeneous distributions [25]. Liu decouples graph features and structural topology in FGL for sharing structural information between clients [6]. Li effectively addresses the non-independent distribution problem in joint graph learning by integrating heterogeneous features of nodes and structures from local data [26].

D. Summary of Related Works

In summary, recent works in GNNs, federated learning and federated graph learning have made significant progress in their respective areas. GNN models like GCN and GIN excel at capturing local graph structures but face challenges in cross-domain scenarios. Federated learning has developed various solutions for data heterogeneity, from meta-learning approaches to cross-correlation optimization. In federated graph learning, researchers have proposed methods to handle non-IID distributions through feature decoupling and structure sharing. However, these existing approaches still struggle with effectively capturing global structural patterns across different domains while maintaining privacy. Our position-aware network aims to address these limitations by explicitly modeling nodes' relative positions and combining both local and global structural information, thus contributing to the advancement of federated graph learning in heterogeneous scenarios.

III. Methodology

In this section, we introduce our proposed Position-Aware Structural Sharing Federated Graph Learning Framework. The framework enables each client to improve its performance on its own graph dataset through FGL with common structural knowledge shared with other clients. We also specifically emphasize the importance of position information for more accurately capturing and exploiting structural features in graphs. Next, we define relevant notations and problems to illustrate how to efficiently acquire structural knowledge of graphs, and discuss how to share this knowledge in federated learning to enhance overall learning.

A. Notation and Problem Definition

1) Graph Neural Networks: A graph is usually denoted as $G = (V, E)$, where $V = \{v_1, \ldots, v_N\}$ represents the set of nodes and $E \subseteq V \times V$ represents the set of edges. In a graph with attributes, each node v_i has an eigenvector $x_i \in \mathbb{R}^d$ forming the node feature matrix $X = \{x_1, \ldots, x_N\}$, where d is the dimension of the features. Graph neural networks process graph data by learning node and edge representations. In GNNs, the feature vector x_v of a node v is used to learn the node-level representation vector h_v and the graph-level representation vector h_G. These representation vectors are crucial for downstream tasks such as node classification, link prediction and graph classification.

2) Federated Learning: The underlying federated learning environment, which contains M clients, each of which holds its own private dataset D_m. The goal of the federated learning framework is to optimize the global objective function as follows:

$$\min_{(w_1, w_2, \cdots, w_M)} \frac{1}{M} \sum_{m=1}^{M} \frac{|D_m|}{N} L_m \left(w_m; D_m\right), \tag{1}$$

Here, N denotes the total number of all client data instances, and L_m and w_m represent the loss function and model parameters of the m-th client, respectively. The conventional approach to traditional federated learning is to train a globally shared model $w = w_1 = w_2 = \cdots = w_M$. FedAvg is a prime example of such an approach, which is implemented by periodically merging model parameters from all clients on a server:

$$\overline{w} \leftarrow \sum_{m=1}^{M} \frac{|D_m|}{N} w_m, \tag{2}$$

This merged average model is then distributed back to each client. However, this approach may have limited effectiveness due to data heterogeneity issues. Recent research attempts to improve the performance of the model on individual client data by introducing personalization strategies in the local training and global aggregation phases.

B. Constructing Graph Structure Embeddings

In traditional GNNs, the representation of a node is implicitly encoded by aggregating the feature information of its neighbours. This approach is effective in many scenarios, but it has some drawbacks when dealing with non-independently identically-distributed (non-IID) graph data, especially in federated learning environments, where feature information-based structural representations may not be able to adequately capture global structural information in the graph due to the fact that data distributions of different clients may vary significantly, thus affecting the overall performance and generalization ability of the model, and making it difficult to federal learning to benefit all clients [27].

To solve this problem, we design two structural encoders, the base structural encoder and the position-aware structural encoder. The base structural encoder captures the local base structural information through the degree-based structural embedding and the random wandering-based structural

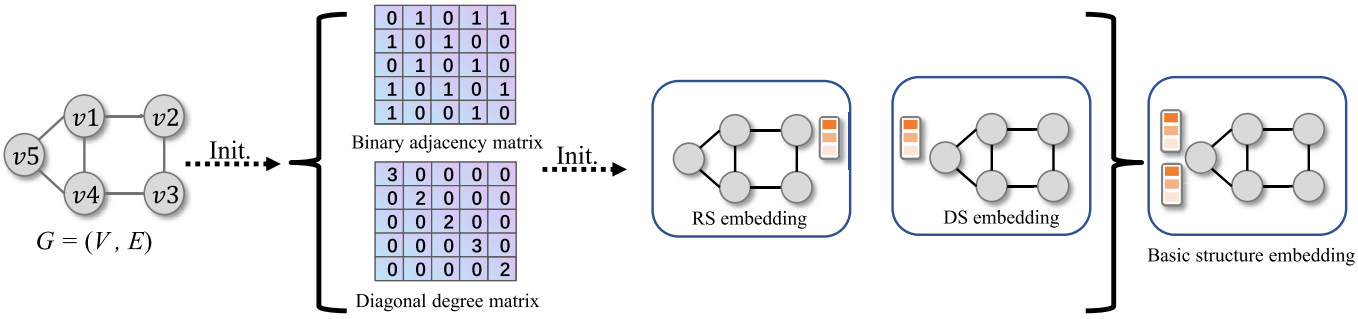


Fig. 1. Basic structural embeddings consisting of random walk-based structural (RS) embedding and degree-based structural (DS) embedding.

embedding, together with the global position-aware structural embedding captured through the position-aware structural encoder to represent the global structural information, and this approach helps different clients in federated learning to better understand the global structural information. This approach can help different clients in federated learning to make better use of global structural information and improve the performance of the model in processing non-independently identically distributed (non-IID) graph data.

1) Base Structure Encoder: In the base structure encoder, we introduce degree-based structure embedding (DS embedding) to capture local structure knowledge. The degree of a vertex is the number of edges connected to that vertex, which is a basic and important feature in graph structure and can be easily extracted from graph data in any domain. DS embedding uses vertex degrees in the form of single-hot encoding, which makes each degree have a unique binary representation and is more efficient in performing computations. In addition, the degree-based single-hot encoding is not only computationally friendly, but also, since degree distributions tend to exhibit certain universal properties (e.g., power-law distributions) in different graph datasets, this allows DS embedding to be efficiently learnt and generalised across different types of graph datasets. For a node v in a graph, its DS embedding is a one-hot encoded vector based on its degree d_v, $\mathbb{I}$ is the identity function, and k_1 is the dimension of the DS embedding, the DS embedding is denoted by

$$\mathbf{s}_v^{\mathrm{DS}} = [\mathbb{I}(d_v = 1), \mathbb{I}(d_v = 2), \cdots, \mathbb{I}(d_v \geq k_1)] \in \mathbb{R}^{\mathrm{k}_1}, \quad (3)$$

At the same time, structural embedding in the random walk is introduced to capture the global structural knowledge computed based on the random walk diffusion process [28]. Specifically, the RS embedding is denoted as

$$\mathbf{s}_v^{\mathrm{RS}} = \left[\mathbf{T}_{ii}, \mathbf{T}_{ii}^2, \cdots, \mathbf{T}_{ii}^{k_2}\right] \in \mathbb{R}^{\mathrm{k}_2}, \quad (4)$$

The random walk transition matrix, denoted as $\mathbf{T}$, is formulated by the binary adjacency matrix $\mathbf{A}$ and the inverse of the diagonal degree matrix $\mathbf{D}$, expressed as $\mathbf{T} = \mathbf{A}\mathbf{D}^{-1}$. In this context, i represents the index of a specific node v. The dimension of the RS Embedding is indicated by k_2.

Finally, the basic structural embedding can be obtained by connecting the local structural knowledge the DS Embedding and the global structural knowledge the RS Embedding, which is denoted as

$$\mathbf{s}_v = \mathrm{concat}[\mathbf{s}_v^{\mathrm{DS}}, \mathbf{s}_v^{\mathrm{RS}}]. \quad (5)$$

As shown in Fig. 1. While the basic structural embedding approach is relatively effective in capturing information about the global structure of the entire graph, this approach does have limitations in capturing information about the specific locations of nodes in the graph. This is mainly due to the fact that the above structural embedding methods mainly focus on the local neighbours of the nodes or the macroscopic features of the overall graph structure rather than the specific positions of the nodes in the graph or their relative relationships with other nodes.

2) The Position-Aware Structural Encoder: For the purpose of better capturing the overall graph structural information, we design a position-aware structural encoder, As shown in Fig. 2, which firstly selects the anchor set by centered selection, then calculates the distance of each node to the anchor set, and finally generates the node embedding by a specific message passing function and aggregation function. This position-aware structural encoder is able to effectively capture the global positional features of the nodes in the graph by considering the positional information of the nodes with respect to a specific anchor set for the purpose of enhancing the representation of node embeddings.

Our goal is to create simple and accurate node embeddings that effectively reflect the positional information in the graph and minimise information distortion during the spatial mapping process. Distortion is in the following definition [7]:

Considering two metric spaces $(\mathcal{V}, d)$ and $(\mathcal{Z}, d')$, and a mapping $f : \mathcal{V} \to \mathcal{Z}$, the function f possesses a distortion of α if for all $u, v \in \mathcal{V}$

$$\frac{1}{\alpha} d(u, v) \leq d'(f(u), f(v)) \leq d(u, v), \quad (6)$$

To achieve this, we select the set of anchor points according to Bourgain's theorem [29] to ensure that the resulting graph embedding accurately preserves the distances between nodes and achieves a low level of distortion.

Bourgain's Theorem: For any finite metric space $(\mathcal{V}, d)$ consisting of n elements, it is possible to embed $(\mathcal{V}, d)$ into $\mathbb{R}^k$ using any l_p metric, where the dimension k is $O(\log^2 n)$, and the embedding's distortion is bounded by $O(\log n)$.

To further improve the position-aware structural encoder's performance, we adopt a centered selection strategy that

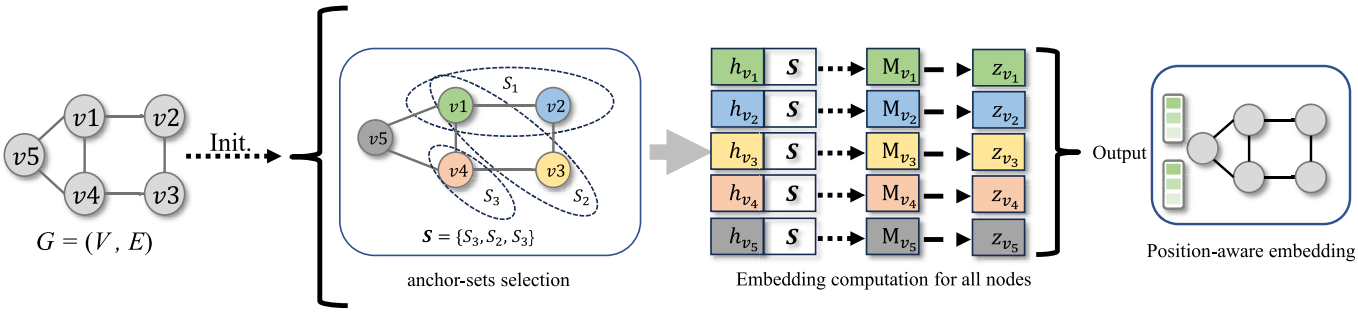


Fig. 2. The graph G is processed through a centralized selection method to select anchor sets of varying sizes $S = \{S_1, S_2, S_3\}$. For each node v_i, the messages $\mathbf{h}_{v_i}$ are computed based on the anchor set S and compiled into the message matrix $\mathbf{M}_{v_i}$. These message matrices are then processed through an aggregation function to produce the position-aware embeddings $\mathbf{z}_{v_i}$ for each node.

combines randomness and graph topology heuristics in choosing the anchor set. In particular, we quantify the importance of a node through feature vector centrality, where the importance measure x_i of a node i can be computed by the following equation:

$$x_i = c \sum_{j=1}^{N} a_{ij} x_j, \tag{7}$$

Here, c is a constant of proportionality, $A = (a_{ij})$ is the adjacency matrix of the graph, and $x = [x_1, x_2, \ldots, x_N]^T$ is a vector representing the importance of all nodes. This metric is based on the structure of the graph, and the importance of each node is determined by the connectivity between nodes. In other words, the importance of a node depends not only on itself but also on the importance of other nodes connected to it.

During each anchor set selection process, we prioritise those nodes with high node importance as anchors based on this importance metric. Such a strategy ensures that the anchor point set can more accurately represent the structural properties of the entire graph, and thus more effectively capture the information about the position of each node relative to the important structure during the position-aware structural encoding process. This approach enhances both the model's structural comprehension and its ability to capture complex topological relationships. The position-aware mechanism enables more accurate representation of node positions, leading to improved performance on complex graph structures.

After that, for each node in the set of anchors S_i, we compute a message using the function F, which combines the node's positional information (computed by the distance between the nodes) and the attribute information (by the node's features). This process can be represented as $F(v, u, \mathbf{h}_v, \mathbf{h}_u)$ for any node u in S_i, where $\mathbf{h}_v$ and $\mathbf{h}_u$ represent the feature embeddings of the node v and node u, respectively.

After message computation, an aggregation function AGG_M is applied to summarise the messages of all nodes in the set of anchor points S_i. This aggregated message is then transformed by a nonlinear operation defined by a weight vector $\mathbf{w} \in \mathbb{R}^r$ and a nonlinear function σ, yielding a scalar value that constitutes the ith dimension of the node embedding $\mathbf{z}_v$. The formal representation of the aggregated message of the anchor set S_i and its conversion to the embedding dimension is as follows:

$$\mathbf{z}_v[i] = \sigma\left(\mathrm{AGG}_M\left(\{F(v, u, \mathbf{h}_v, \mathbf{h}_u) \mid u \in S_i\}\right) \cdot \mathbf{w}\right), \tag{8}$$

Output embeddings $\mathbf{z}_v$ each embedding dimension carries key information used to distinguish between structurally similar but differently located nodes in the graph. Even if the dimensions of the embeddings are replaced, the embeddings still retain the same positional information as long as the consistency of the dimension replacement of all node embeddings is maintained. This design allows the embeddings to not only reflect the local structural properties of the nodes, but also capture the global position of the nodes in the entire graph structure, thus providing richer information for graph-based prediction tasks.

The base structure encoder captures both local and global structural knowledge through degree-based and random-walk embeddings. This dual-embedding approach ensures that the encoder effectively generalizes across diverse graph domains, addressing the limitations of previous methods such as FedStar, which primarily focus on local structural features. Furthermore, the position-aware structural encoder introduces a novel centralized anchor selection strategy, enhancing the embedding's ability to distinguish nodes with similar local neighborhoods but differing global positions. These innovations significantly improve the representation power and generalization ability of the model in Non-IID federated settings.

C. Enhanced Feature-Structure Decoupled GNN Architecture

In our previous work, we employed two structural encoders to acquire structural knowledge of graphs, which focus on capturing topological and connectivity patterns in graphs, thus helping us understand the overall structural properties of graphs. In the existing FGL framework, when client data originates from different domains, directly constructing and sharing feature-based encoders can lead to misaligned representations. This misalignment occurs due to differences in feature distributions and data structures across domains, ultimately impairing the model's personalization capabilities.

Inspired by LSPE (Learnable Structural and Positional Encodings) [5] and FedStar [6], in order to be able to better adapt to graph data from different domains and optimize the model performance so that it can be used in federated

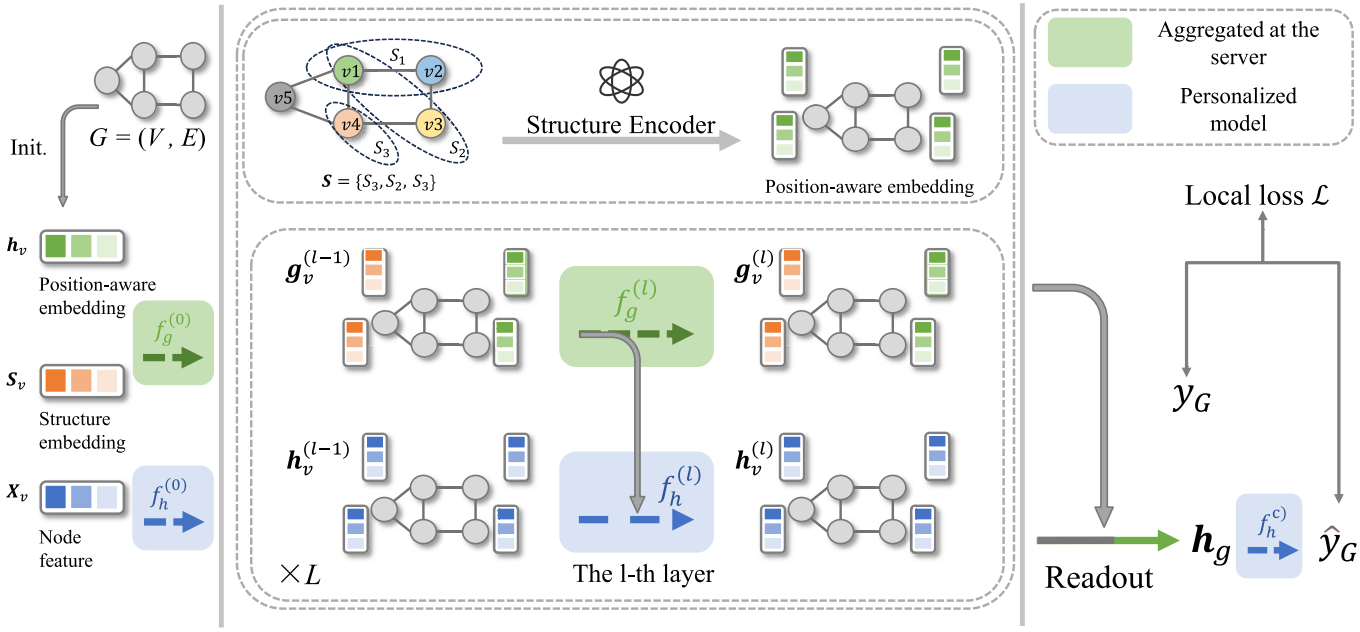


Fig. 3. The overall layout of our proposed GNN architecture: the the blue colored boxes show models that are locally trained and customized for each client, and the green boxes represent models that are aggregated on the server-side (which allows for the sharing of knowledge across clients).

learning environments to effectively handle cross-domain non-independent identically distributed (Non-IID) graph of data in a FL environment. We propose an enhanced two-channel graph neural network framework that separately learns attribute and structure knowledge through two channels in parallel.

Under this architecture, the feature encoder in the feature channel acts directly on the original node features to learn the attribute information of every node in the graph by the aggregated attribute and structural information of neighboring nodes, while the structural encoder in the structural channel is constructed to learn the structural embedding information of the graph through the structural encoder. This feature-structure separation architecture ensures that the network can capture both attribute feature of the nodes and their structural positions in graph, and the model can extract useful graph structural knowledge while retaining each domain-specific information, enhancing its adaptability and prediction performance on diverse data sources. And we introduce a position-aware function in the structural encoder, which not only can capture the structure features of graphs, but also generate more accurate embedding representatives based on the positions of nodes in the graphs.

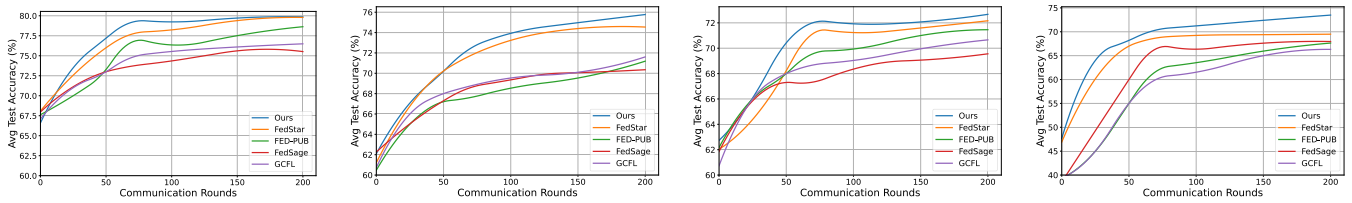
Fig. 3 illustrates the overall architecture of our proposed graph neural network (GNN), which consists of three main components: (i) Initial feature and structure linear transformation layers $f_h^{(0)}$ and $f_g^{(0)}$, which linearly transform input features and structure embeddings via learnable parameters $w_h^{(0)}$ and $w_g^{(0)}$ respectively; (ii) A stack of L enhanced two-channel GNN layers, each denoted by $f_h^{(l)}$ and $f_g^{(l)}$ for $l \in \{1, \ldots, L\}$, learned through parameters $w_h^{(l)}$ and $w_g^{(l)}$; (iii) A classifier $f_h^{(C)}$, implemented by parameter $w_h^{(C)}$ to classify nodes within the graph.

Consider the graph G constituted by a set of nodes V and edges E, with each node v characterized by a feature vector $\mathbf{x}_v$ and an initial structural embedding $\mathbf{s}_v$. First, we obtain the position-aware embedding $\mathbf{h}_v$ through the position-aware structural encoder, and then convert the structural embedding consisting of $\mathbf{s}_v$ and $\mathbf{h}_v$ to the corresponding input embedding $\mathbf{g}_v^{(0)}$ through the linear layer $\mathbf{g}_v^{(0)}$, and similarly the structural embedding of $\mathbf{x}_v$ through $f_h^{(0)}$ is transformed into the input embedding $\mathbf{g}_v^{(0)}$. This process of linear transformation aligns the feature and structural inputs into vectors with uniform dimensions. Subsequently, the resultant embeddings, $h_v^{(0)}$ for features and $g_v^{(0)}$ for structures, are inputted into the L-layered GNN for subsequent refinement.

After the stacking of the L ayers, the network outputs the hidden feature embeddings $\{\mathbf{h}_v^{(L)} : v \in V\}$ and the structural embeddings $\{\mathbf{g}_v^{(L)} : v \in V\}$ of all nodes. They are concatenated together to form a richer node-level representation, and then in order to extract global information from the node-level information that can represent the whole graph, the concatenated node-level embeddings are further converted into graph-level embeddings $\mathbf{h}_G$ representing the whole graph G through a readout function. Finally, the model is optimized using a cross-entropy loss $\mathcal{L}m$ for each client m with a set of learnable parameters $w_m = \{w_{h,m}, w_{g,m}\}$, where $w_{h,m}$ comprises the parameters of the feature encoder and the classifiers, and $w_{g,m}$ comprises the parameters of the structural encoder. The parameters are updated through an optimization step in the local training process with the update rule of

$$w_{h,m}, w_{g,m} = \underset{w_{h,m}, w_{g,m}}{\arg\min}\, \mathcal{L}_m(w_m; \mathcal{D}_m). \tag{9}$$

That is, finding the combination of parameters that minimizes the loss function.



(a) TRFC (b) TRFC-RD-POI (c) TRFC-RD-SOC (d) TRFC-RD-SNS-VIS

Fig. 4. Accuracy graphs for our method and three FGL approaches over communication rounds, with subplots for unique cross-dataset (a) or cross-domain (b,c,d) non-IID conditions.

The enhanced two-channel GNN architecture effectively decouples feature learning and structural embedding, addressing the feature heterogeneity issues commonly encountered in Non-IID federated learning environments. Compared to single-channel GNN architectures, this design ensures that domain-specific feature information is preserved while leveraging shared structural knowledge for improved cross-domain generalization. By incorporating position-aware embeddings, the structural channel further refines the representation of graph structures, enabling more accurate predictions in federated graph learning scenarios.

D. Federated Graph Structure Knowledge Sharing

Thanks to the enhanced two-channel graph neural network framework, we can capture generalized structural information, and in order to mitigate the problem of prior heterogeneity of graph data that may lead to unstable model training and performance degradation, we share the captured generalized structural information among clients using federated learning.

Specifically, we decouple the local model parameters w_m of the mth client into a feature submodel $w_{h,m}$ and a structural submodel. The structural submodel $w_{g,m}$ is then shared in a federated learning framework while keeping the feature submodel $w_{h,m}$ trained locally.

Our training methodology mirrors that of FedAvg [14]. Initially, each client fine-tunes its local model with its proprietary data. Upon completion of local training, every client dispatches the parameters of its structural submodel $w_{g,m}|_{m=1}^{M}$ to the central server. It is crucial to note that this procedure does not entail the transmission of any raw data or parameters of the feature submodel. The central server then executes a weighted mean aggregation of the parameters submitted.

$$\overline{w}_g = \sum_{m=1}^{M} \frac{|\mathcal{D}_m|}{N} w_{g,m}, \tag{10}$$

where, $|\mathcal{D}_m|$ represents the count of graphs contained within the local dataset of client m, and N indicates the aggregate sum of graphs from all participating clients. The server dispatches the consolidated global structural model parameters $\overline{w}_g$ back to the individual clients. These clients then integrate the received global structural model parameters to update their respective local structural submodels, setting the stage for the forthcoming training cycle.

With such an approach, federated graph structural knowledge sharing not only enhances the generalization ability of models, but also maintains data privacy. It allows valuable structural information to be exchanged between different clients without involving the original data, thus enabling cross-domain knowledge sharing and model collaboration.

IV. Experiments

A. Experiment Setting

1) Dataset: We adopted the experimental setup described in FedStar, utilizing 16 publicly available graph classification datasets spanning four distinct transportation-related fields: basic traffic networks (including datasets like California Road Network, PeMS subsets, METR-LA, PEMS-BAY, PEMS03, PEMS04, PEMS07), road networks with points of interest (California Road Network with POIs, PeMS with POIs), traffic-related social media data (Waze incidents, X traffic posts), and sensor-visual integrated traffic data (highD dataset, UTD19, TrafficVision), as inspired by real-world transportation datasets.

These datasets exhibit diverse graph structural characteristics across domains. For basic traffic network datasets, graphs are typically moderate in size (50-200 nodes) with sparse connectivity patterns representing road segments and intersections. Road network datasets with POIs contain additional nodes (100-500 nodes) representing points of interest like gas stations or restaurants, with varying edge density (average degree 3-5). Traffic-related social media datasets feature dynamic connectivity (average degree 6-10) with event-driven structures. Sensor-visual integrated datasets have multi-modal graph patterns, combining grid-like visual data and time-series sensor readings with consistent node degrees.

To explore and validate the performance of our method in processing non-IID data, we designed four test scenarios reflecting the heterogeneity of data across transportation contexts:

- **TRAFFIC**: A cross-dataset scenario covering seven basic traffic network datasets, handling road graphs with similar structural patterns but varying sizes (average 50-200 nodes) and connectivity.
- **TRAFFIC-ROAD-POI**: A cross-domain scenario combining basic traffic networks and road networks with POIs, merging moderate-sized road graphs with larger networks including points of interest and introducing significant size variation (50-500 nodes) and different connection patterns.
- **TRAFFIC-ROAD-SOCIAL**: A cross-domain scenario containing basic traffic networks, road networks with

POIs, and traffic-related social media data, further increasing structural diversity by including dynamic event-driven networks, spanning from sparse road graphs to dense social interaction networks.

- **TRAFFIC-ROAD-SENSOR-VISION**: A cross-domain scenario combining basic traffic networks, social media data, and sensor-visual integrated datasets, representing maximum structural heterogeneity with road networks, event-driven structures, and multi-modal sensor-visual graphs.

In these scenarios, the datasets owned by each client are randomly divided into three parts: 80% for training, 10% for validation, and 10% for testing.

2) Baseline Models and Implementation Details: We comprehensively evaluate our proposed method against seven representative baseline approaches spanning different federated learning paradigms. For fair comparison, we maintain consistent training configurations across all methods where applicable.

Our baseline methods can be categorized into two groups: basic federated learning methods and graph-specific federated methods. The basic federated learning methods include Local, where clients train independently without communication; FedAvg [14], the standard federated averaging algorithm; FedProx [16], which addresses client heterogeneity through proximal regularization ($\mu = 0.01$); and FedPer [30], which enables personalization by selective parameter sharing.

For graph-specific methods, we compare against FedSage [31], which extends GraphSage to federated settings with a three-layer architecture (64 hidden units per layer) and mean aggregation; GCFL [4], a clustering-based approach using parameters $\epsilon_1 = 0.5$ and $\epsilon_2 = 1.0$ as recommended in their work; FedStar [6], which addresses non-IID data through feature-structure decoupling; and FED-PUB [32], which proposes a personalized subgraph federated learning framework based on functional embedding, calculating model similarity through random graph input and combining sparse masks to achieve parameter localization.

For our proposed method, we employ a multi-component architecture comprising three key elements: a GCN-based structural encoder [33], a GIN-based feature encoder [34], and a P-GNN-based position-aware structural encoder [7], each configured with 64 hidden units. The embedding dimensions are carefully selected, with degree-based (k_1) and random-walk based (k_2) structural embeddings set to 8 dimensions each, while the position-aware structural embedding (k_3) uses 16 dimensions.

To ensure reproducibility and fair comparison, we maintain consistent training configurations across all experiments. Each client performs one local training epoch with a batch size of 128 before model aggregation. We employ the Adam optimizer with a learning rate of 0.001 and weight decay coefficient of 5e-4. The federated learning process continues for 200 communication rounds. Statistical significance is ensured through five independent runs with different random seeds, with results reported as means and standard deviations. All implementations utilize the PyTorch framework, with experiments conducted on NVIDIA RTX 4090 GPUs.

B. Experimental Results and Analysis

1) Performance Comparison: Extensive experiments were conducted across four non-IID scenarios to evaluate our proposed method against baseline approaches, with results summarized in Table I. The experimental results demonstrate the superiority of our method while revealing key challenges in handling non-IID graph data.

In the single-domain TRAFFIC scenario containing road graphs with similar structural patterns (50-200 nodes), our method achieves 79.90% accuracy with a 4.50% improvement over local training. This performance gain indicates effective handling of traffic network characteristics through our structural embedding strategy.

For cross-domain scenarios with increasing structural heterogeneity, our method maintains robust performance:

- TRAFFIC-ROAD-POI scenario combines basic traffic networks with road networks including points of interest, introducing significant size variations (50-500 nodes) and diverse connection patterns. Our method achieves a 4.75% gain through effective alignment of heterogeneous structural features.
- TRAFFIC-ROAD-SOCIAL further increases structural diversity by incorporating traffic-related social media data, where our method demonstrates a 3.25% improvement via flexible parameter aggregation.
- In the most heterogeneous TRAFFIC-ROAD-SENSOR-VISION scenario spanning road networks, social media data, and sensor-visual integrated graphs, our method achieves a 6.10% gain through optimized multi-view structural representations.

The consistent standard deviations (1.96-2.90) across scenarios demonstrate the stability of our approach. Traditional federated methods like FedAvg and FedProx show performance degradation on heterogeneous graph data, while graph-specific methods (FedSage, GCFL) achieve moderate improvements through structural exploitation. The superior performance of our method, particularly compared to random anchor selection (Ours[RS]), validates the effectiveness of our centralized selection strategy in capturing domain-invariant structural patterns.

2) Convergence Analysis: As shown in Fig. 4, we analyze the convergence behavior across four non-IID scenarios through test accuracy curves averaged over randomized trials. Our method demonstrates superior convergence properties in all scenarios, achieving faster convergence and higher accuracy compared to baseline methods. The enhanced convergence performance can be attributed to our position-aware structural embeddings, which facilitate more efficient encoder alignment across heterogeneous domains. The narrow standard deviation bands in the convergence curves demonstrate that our method achieves both faster convergence and better stability compared to baseline approaches, highlighting its scalability in handling non-IID graph data across different domains.

3) Effects of Different Structure Embeddings: To explore the specific contribution of position-aware structural embeddings to federated graph learning performance, we designed a comprehensive ablation study with different structural embedding components. We examine three key aspects: (1) the impact of different encoder architectures, (2) the contribution

TABLE I

PERFORMANCE IS EVALUATED ACROSS VARIOUS FEDERATED GRAPH CLASSIFICATION SCENARIOS, WHERE EACH SCENARIO ENCOMPASSES DISTINCT DATASETS HELD BY SEPARATE CLIENTS. THESE DATASETS ORIGINATE FROM EITHER A SINGLE DOMAIN OR MULTIPLE DOMAINS

Setting (# domains)	TRFC(1)		TRFC-RD-POI(2)		TRFC-RD-SOC(3)		TRFC-RD-SNS-VIS(3)	
# datasets	7		10		13		9	
Accuracy	avg.	avg. gain	avg.	avg. gain	avg.	avg. gain	avg.	avg. gain
Local	75.40±2.24	-	71.05±1.23	-	69.35±3.03	-	66.95±2.82	-
FedAvg	75.23±2.02	-0.17	70.62±2.71	-0.43	68.90±2.14	-0.45	64.83±2.75	-2.12
FedProx	75.33±1.98	-0.07	70.72±2.28	-0.33	69.18±2.61	-0.17	65.15±2.03	-1.80
FedPer	77.12±3.34	1.72	71.94±1.95	0.89	69.40±2.90	0.05	62.20±3.74	-4.75
FedSage	75.93±1.87	0.53	70.31±1.85	-0.74	69.58±2.17	0.23	67.98±1.89	1.03
GCFL	76.45±1.25	1.05	71.63±2.18	0.58	70.68±1.86	1.33	66.28±2.34	-0.67
FED-PUB	78.60±1.90	3.20	74.15±2.05	3.10	71.35±1.96	2.00	69.61±2.13	2.66
FedStar	79.82±2.42	4.42	74.57±2.48	3.52	72.13±2.45	2.78	69.52±1.83	2.57
Ours[RS]	**79.73**±2.53	**4.33**	**75.10**±2.62	**4.05**	**72.47**±2.35	**3.12**	**72.58**±2.40	**5.63**
Ours	**79.90**±2.78	**4.50**	**75.80**±2.90	**4.75**	**72.60**±2.82	**3.25**	**73.05**±1.96	**6.10**

TABLE II

COMPREHENSIVE ABLATION STUDY ON MODEL COMPONENTS AND STRUCTURE EMBEDDINGS. GCN: GRAPH CONVOLUTIONAL ENCODER, GIN: GRAPH ISOMORPHISM ENCODER, P-GNN: POSITION-AWARE ENCODER. DS: DEGREE-BASED STRUCTURE, RW: RANDOM-WALK BASED STRUCTURE, PS: POSITION-AWARE STRUCTURE. RESULTS SHOW ACCURACY (%) ON DIFFERENT SETTINGS

Architecture	**Embeddings**	**Setting (# domains)**			**Avg. Drop**
Components	Feat.	TRFC-RD-POI(2)	TRFC-RD-SOC(3)	TRFC-RD-SNS-VIS(3)	vs. Full
GCN+GIN+P-GNN	DS+RW+PS	**75.80±3.53**	**72.60±2.82**	**73.05±1.96**	-
GIN+P-GNN	DS+RW+PS	73.95±3.23	70.85±2.63	71.28±2.11	-1.8%
GCN+P-GNN	DS+RW+PS	73.48±3.31	70.50±2.76	70.95±2.06	-2.2%
GCN+GIN	DS+RW+PS	72.62±3.40	69.90±2.90	70.15±2.17	-3.1%
GCN+GIN+P-GNN	DS only	74.57±2.48	72.18±2.45	69.52±1.83	-1.9%
GCN+GIN+P-GNN	RW only	74.10±2.33	71.37±3.09	69.00±2.22	-2.3%
GCN+GIN+P-GNN	PS only	73.85±2.83	71.05±2.74	68.82±2.29	-2.6%

TABLE III

PERFORMANCE VARIATIONS IN A CROSS-DATASET NON-IID SETTING (TRAFFIC) WITH DIFFERING NUMBERS OF LOCAL TRAINING EPOCHS

Epochs	1	2	3	4
FedAvg	75.23±2.02	**76.62±2.70**	76.93±3.28	76.33±3.34
GCFL	**76.45±1.25**	76.20±2.97	75.85±2.63	76.30±2.12
FedStar	79.82±2.42	80.23±1.87	79.93±2.94	**80.60±2.46**
Ours	79.90±2.78	80.13±2.70	80.00±2.33	**81.05±2.34**

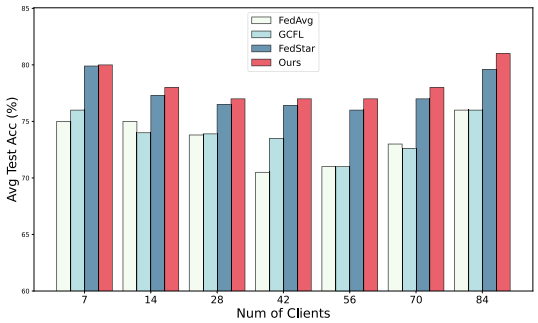


Fig. 5. Variability in performance across different client counts in the non-IID TRAFFIC dataset scenario.

of each embedding type, and (3) the interaction between different structural features.

Our encoder architecture experiments show that removing any single component (GCN, GIN, or P-GNN) leads to performance degradation (-1.8% to -3.1%), confirming the necessity of our multi-component design. Feature interaction analysis reveals that the combination of degree-based and random-walk embeddings provides complementary structural information, improving performance by 2.3% compared to using either alone.

As shown in Table II, the experiments were performed in three different non-IID scenarios. The results show that the model using the position-aware structural embedding achieves significant performance improvements in all scenarios. The analysis shows that the position-aware structural embedding is more effective in capturing and utilizing the relative position information between nodes in the graph than the traditional structural embedding.

Furthermore, these experimental results highlight the importance of integrating structural and positional information when designing federated learning algorithms. Our findings provide new directions for future research in this area to further explore and optimize position-aware mechanisms to enhance the overall performance of federated graph learning.

4) Varying Local Epochs: In federated learning, increasing local training epochs before global aggregation is crucial for reducing communication costs in real-world deployments. As shown in Table III, our method demonstrates stronger performance gains with increased local epochs compared to baseline approaches. This advantage indicates that our method not only effectively handles non-IID data challenges but also achieves better communication efficiency. The consistent performance improvement across different epoch settings further validates our framework's scalability and practical utility in resource-constrained federated environments.

5) Varying Numbers of Clients: To evaluate scalability, we tested our method by increasing client numbers from 7 to

84 in the TRAFFIC setting through dataset subdivision. As shown in Fig. 5, while all methods initially show slight performance degradation due to increased client heterogeneity, our approach maintains stable performance as client numbers grow. The performance gradually improves with reduced dataset sizes per client, demonstrating our method's effectiveness in handling distributed graph data at scale. These results validate our framework's applicability in large-scale federated environments with heterogeneous data distributions.

V. Conclusion

In this paper, we propose a position-aware structure-sharing federated graph learning framework to overcome the challenge of diversity in non-independently identically distributed data and graph structures. It enables graph federated learning systems to exploit the structural properties of graph data more effectively by integrating the location information. By sharing location embeddings in the federated framework, participating clients can optimize the processing of local data and leverage the structural knowledge of other clients to enhance the overall learning effect. The experimental results show that our approach can get superior performance compared to existing techniques on different datasets, especially in processing non-IID data.

References

[1] F. Chen, P. Li, T. Miyazaki, and C. Wu, "FedGraph: Federated graph learning with intelligent sampling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1775–1786, Aug. 2022.

[2] T. Zhang et al., "An adaptive federated relevance framework for spatial temporal graph learning," *IEEE Trans. Artif. Intell.*, vol. 5, no. 5, pp. 2227–2240, May 2022.

[3] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1150–1160.

[4] H. Xie, J. Ma, L. Xiong, and C. Yang, "Federated graph classification over non-IID graphs," in *Proc. 35th Adv. Neural Inf. Process. Syst.*, 2021, pp. 18839–18852.

[5] V. P. Dwivedi, A. T. Luu, L. Thomas, Y. Bengio, and X. Bresson, "Graph neural networks with learnable structural and positional representations," Jan. 2021, *arXiv:2110.07875*.

[6] Y. Tan, Y. Liu, G. Long, J. Jiang, Q. Lu, and C. Zhang, "Federated learning on non-IID graphs via structural knowledge sharing," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 8, 2023, pp. 9953–9961.

[7] J. You, R. Ying, and J. Leskovec, "Position-aware graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7134–7143.

[8] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[9] Y. Liu, K. Ding, H. Liu, and S. Pan, "GOOD-D: On unsupervised graph out-of-distribution detection," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, Feb. 2023, pp. 339–347.

[10] Z. Xie, Y. Huang, D. Yu, R. M. Parizi, Y. Zheng, and J. Pang, "FedEE: A federated graph learning solution for extended enterprise collaboration," *IEEE Trans. Ind. Informat.*, vol. 19, no. 7, pp. 8061–8071, Jul. 2023.

[11] Y. Tan, G. Long, J. Ma, L. Liu, T. Zhou, and J. Jiang, "Federated learning from pre-trained models: A contrastive learning approach," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 19332–19344.

[12] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, Jan. 2021, Art. no. 106775.

[13] J. Zhang et al., "Federated learning with label distribution skew via logits calibration," in *Proc. 39th Int. Conf. Mach. Learn.*, vol. 162, Jul. 2022, pp. 26311–26329.

[14] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, Apr. 2017, pp. 1273–1282.

[15] S. Wang, M. Lee, S. Hosseinalipour, R. Morabito, M. Chiang, and C. G. Brinton, "Device sampling for heterogeneous federated learning: Theory, algorithms, and implementation," in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.

[16] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. 3rd Mach. Learn. Syst. Conf.*, 2020, pp. 429–450.

[17] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, Jul. 2020, pp. 5132–5143.

[18] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12878–12889.

[19] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," in *Proc. NeurIPS Conf.*, vol. 33, Dec. 2020, pp. 3557–3568.

[20] W. Huang, M. Ye, and B. Du, "Learn from others and be yourself in heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10143–10153.

[21] M. Yu, O.-K. Kwon, and S. Oh, "Addressing client heterogeneity in synchronous federated learning: The CHAFL approach," in *Proc. IEEE 29th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2023, pp. 2804–2805.

[22] D. Caldarola, M. Mancini, F. Galasso, M. Ciccone, E. Rodola, and B. Caputo, "Cluster-driven graph federated learning over multiple domains," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2021, pp. 2749–2758.

[23] R. Ye, Z. Ni, F. Wu, S. Chen, and Y. Wang, "Personalized federated learning with inferred collaboration graphs," in *Proc. Int. Conf. Mach. Learn.*, vol. 202, Jul. 2023, pp. 39801–39817.

[24] K. Wang, Y. Liu, Q. Ma, and Q. Z. Sheng, "MulDE: Multi-teacher knowledge distillation for low-dimensional knowledge graph embeddings," in *Proc. Web Conf.*, Apr. 2021, pp. 1716–1726.

[25] C. Chen, Z. Xu, W. Hu, Z. Zheng, and J. Zhang, "FedGL: Federated graph learning framework with global self-supervision," *Inf. Sci.*, vol. 657, Feb. 2024, Art. no. 119976.

[26] B. Li, Y. Ma, Y. Liu, H. Gu, Z. Chen, and X. Huang, "Federated learning on distributed graphs considering multiple heterogeneities," in *Proc. ICASSP - IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 5140–5144.

[27] X. Zhang, Q. Wang, Z. Ye, H. Ying, and D. Yu, "Federated representation learning with data heterogeneity for human mobility prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 6111–6122, Jun. 2023.

[28] Y. Liu, Y. Zheng, D. Zhang, V. C. S. Lee, and S. Pan, "Beyond smoothing: Unsupervised graph representation learning with edge heterophily discriminating," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 4, pp. 4516–4524.

[29] J. Bourgain, "On Lipschitz embedding of finite metric spaces in Hilbert space," *Israel J. Math.*, vol. 52, nos. 1–2, pp. 46–52, 1985.

[30] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," 2019, *arXiv:1912.00818*.

[31] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph federated learning with missing neighbor generation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, Jan. 2021, pp. 6671–6682.

[32] J. Baek, W. Jeong, J. Jin, J. Yoon, and S. J. Hwang, "Personalized subgraph federated learning," in *Proc. ICML Workshop*, Jan. 2022, pp. 1396–1415.

[33] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," Apr. 2017, *arXiv:1609.02907*.

[34] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks," Oct. 2018, *arXiv:1810.00826*.